

A Locally-Hosted Retrieval-Augmented Generation Pipeline for University Course Material: Design, Implementation, and Evaluation

Yogeshvar Reddy

Department of Electrical Engineering and Computer Science
The Pennsylvania State University, University Park, PA, USA
yv5381@psu.edu

Abstract—Modern large language models display strong fluency but cannot reliably ground their answers in a specific body of source material, which limits their usefulness when students study from a particular university course. We present an end-to-end retrieval-augmented generation pipeline that ingests a learning-management-system corpus and exposes it through a multi-turn chat interface backed by a locally-hosted language model. The pipeline scrapes 213 course items from Canvas via its REST API, normalizes them to portable document format, chunks the resulting text on a per-page basis, embeds each chunk into a 384-dimensional vector with a MiniLM sentence-transformer, and indexes 1,442 vectors in a FAISS inner-product structure. At query time the top-K most similar chunks are injected into a system-prompted Llama 3.1 8B model running under Ollama on commodity hardware. The system enforces inline bracketed citations to source files and refuses to answer questions outside the indexed corpus. We describe the design at each stage, with particular emphasis on the retrieval-and-generation engine; report retrieval and latency measurements on an Apple M3 laptop; and discuss the trade-offs of an entirely on-device deployment. The result is a private, offline study assistant that answers course questions with verifiable references in roughly eleven seconds per turn.

Index Terms—retrieval-augmented generation, vector search, FAISS, sentence embeddings, local large language models, learning management systems, citation grounding, course assistants.

I. INTRODUCTION

Large language models (LLMs) have become a default aid for university students. They can explain concepts, generate worked examples, and rephrase technical material in different registers, all in seconds. Yet a generic LLM hallucinates: when asked about a specific course, it may invent definitions, attribute false properties to algorithms, or cite textbooks the instructor never assigned. For graded study, this is a serious correctness problem and a serious learning problem; the student has no way to verify the model's claim against the actual lecture notes.

Retrieval-augmented generation (RAG) addresses this gap by separating *what the model knows* from *what the model says*. A retrieval component first locates passages in an authoritative corpus that are relevant to the user's question; the LLM is then asked to answer using only those passages, with explicit citations back to them. The model becomes a fluent surface over a source of truth rather than the source of truth itself, and the answers become verifiable.

This paper describes the design and implementation of a complete, locally-hosted RAG system over the corpus of a single Penn State course (CMPSC 132 — Programming and Computation II: Data Structures) hosted in the Canvas learning management system. Our system runs entirely on a student's laptop: data collection, indexing, retrieval, and generation are all offline, and no course content leaves the device. The contributions are: (1) a robust Canvas-API scraper that exports heterogeneous course items to a single normalized format; (2) a per-page chunking and embedding pipeline that preserves citation precision; (3) a stateless context-injection design that keeps multi-turn chat history clean; (4) a citation-disciplined system prompt that produces

verifiable answers; and (5) two front-ends, a command-line REPL and a Gradio-based browser UI, that share a single retrieval core.

II. SYSTEM OVERVIEW

The system is organized into three execution phases, illustrated in Fig. 1.

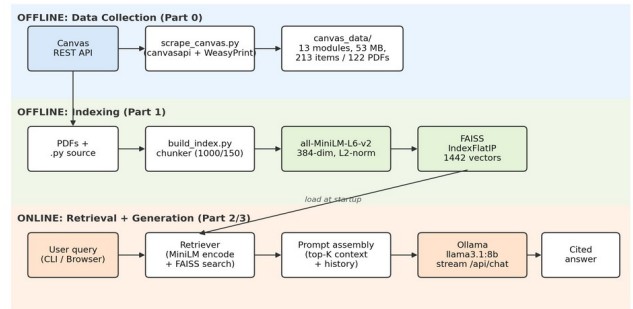


Fig. 1. End-to-end system architecture, organized by execution phase.

Fig. 1. End-to-end system architecture. Offline phases (top two lanes) build on-disk artifacts that the online phase (bottom lane) consumes per query.

The *data-collection* phase uses the Canvas REST API to enumerate every item in the target course and write a structured tree of PDFs and metadata to disk. The *indexing* phase walks that tree, extracts text on a per-page basis, chunks it into overlapping windows, embeds each window with a sentence-transformer, and stores the resulting vectors in a FAISS index. Both phases are run once before serving and produce only on-disk artifacts. The *online* phase loads those artifacts into memory at startup and serves user queries

by retrieving the top-K most similar chunks, injecting them into a prompt, and streaming a cited response from a local LLM. Three front-end programs share the online code path: a one-shot retrieval CLI for sanity-checking the index, a multi-turn chat REPL, and a browser application built with Gradio.

All three online programs depend on a single *Retriever* class that wraps the embedding model and the FAISS index. Centralizing retrieval logic in one module ensures that any tweak to chunking, normalization, or scoring takes effect uniformly across the CLI, the chat REPL, and the web UI without code duplication.

III. DATA COLLECTION FROM CANVAS

Canvas exposes a comprehensive REST API that returns structured JSON for every navigable item in a course: pages, files, assignments, discussions, quizzes, external URLs, and section dividers. We chose this interface over HTML scraping because the schema is stable across Canvas versions, authentication is straightforward via per-user access tokens, and rate limiting is not encountered in practice for single-course exports. Our scraper is written in Python and uses the *canvasapi* wrapper [1], [2].

Each item type follows a different export route. Wiki *Pages* are downloaded as HTML and rendered to PDF with WeasyPrint [3]; the resulting PDFs preserve embedded images and code listings. *Files* are downloaded verbatim, so original PDFs and Python source files retain their native extensions and content. *Assignments*, *Quizzes*, and *Discussions* are exported as a description PDF plus a JSON sidecar that captures structured metadata (due dates, points, allowed attempts). *External URLs* are recorded only as references; we do not crawl beyond the LMS. *Sub-headers* become section dividers in a per-module index file.

WeasyPrint's default URL fetcher uses the Python standard library's TLS stack, which on macOS does not honor the system trust store. Embedded images served from the Canvas CDN therefore failed during HTML-to-PDF rendering with certificate-verification errors. We replaced the default fetcher with a *requests*-backed implementation that uses the certifi CA bundle and forwards the Canvas bearer token on same-host requests. This single replacement eliminated the TLS issue and unblocked rendering of every page that referenced authenticated Canvas media.

The complete export of CMPSC 132 produced 13 module folders containing 213 items in 53 MB on disk: 78 plain files, 47 wiki pages, 46 sub-headers, 16 external URL stubs, 13 quiz descriptions, 12 assignment descriptions, and one discussion description. Of these, 122 items rendered to PDF; the remainder are pass-through files or metadata-only entries. Zero items produced error records during export. The module tree, with one folder per module and per-module index files, is the corpus for everything that follows.

IV. INDEXING PIPELINE

The indexing pipeline turns the static *canvas_data/* tree into three on-disk artifacts that the online system consumes: a

JSON-Lines file of chunk metadata, a NumPy array of normalized embedding vectors, and a FAISS index file.

A. Per-Page Text Extraction

We extract text from PDFs with pypdf [4], reading one page at a time and tagging each emitted chunk with its source path and page number. Per-page extraction has a single but important downstream consequence: when the system later cites a chunk, it can produce a precise location such as "*slides_analysis_of_algorithms.pdf (page 11)*" rather than a whole-document reference. The .py source files in the corpus are read as plain UTF-8 text without page metadata.

B. Recursive Chunking with Overlap

Within each page (or each .py file), text is split using a recursive paragraph-first chunker. The chunker first attempts to split on double newlines, falls back to single newlines, then to sentence boundaries, and finally to a hard character limit. Each chunk is at most 1,000 characters; consecutive chunks overlap by 150 characters. The overlap is deliberate: a chunk boundary that bisects a definition or a code example would otherwise leave the relevant concept incomplete in both adjacent chunks. The overlap is large enough to preserve a typical sentence but small enough to keep the index from doubling in size.

C. Sentence-Transformer Embeddings

We embed each chunk with all-MiniLM-L6-v2 [5], [6], a 22 M parameter MiniLM-style sentence-transformer that produces 384-dimensional vectors. The model is small enough to run on CPU but well-trained for general-domain semantic similarity, and it is the most widely-used baseline embedding model in the sentence-transformers ecosystem. Vectors are L2-normalized at encode time so that subsequent inner-product search is mathematically equivalent to cosine similarity.

The output of indexing on the CMPSC 132 corpus is 1,442 chunks occupying 5.4 MB across all artifacts: 2.2 MB of float32 vectors in NumPy format, 1.2 MB of chunk-text JSON-Lines, a 2.2 MB FAISS index file, and a small JSON metadata file recording the embedding-model name, vector dimensionality, chunk parameters, and build timestamp. The build runs in well under a minute on a single laptop with no GPU.

V. RETRIEVAL AND GENERATION

This section, which describes the core of the system, is structured around the per-turn data flow shown in Fig. 2. We discuss in turn: how the FAISS index is constructed, how queries are resolved against it, how prompts are assembled, how generation is streamed from a local LLM, and how multi-turn history is managed without polluting future prompts.

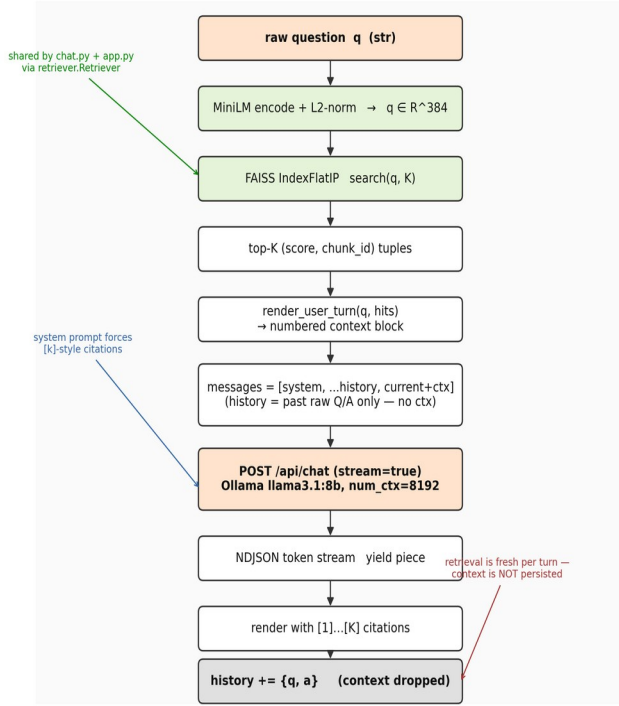


Fig. 2. Per-turn retrieval-and-generation pipeline. Side annotations highlight three design constraints: shared retrieval logic, citation enforcement in the system prompt, and stateless context injection.

A. Vector Store Construction with FAISS

We chose FAISS [7] for the vector store because it is a mature, MIT-licensed library with bindings in both Python and C++ and a wide selection of index types ranging from exact flat search to approximate IVF and HNSW variants. For our corpus size of 1,442 vectors, exhaustive search is more than sufficient: scoring a single query against the entire index requires 1,442 inner products on 384-dimensional float32 vectors, which completes in well under a millisecond and does not benefit from the cost-accuracy trade-off of approximate methods.

We therefore use IndexFlatIP, an exact index that scores by inner product. Given a row-normalized corpus matrix $D \in \mathbb{R}^{N \times d}$ and a normalized query $q \in \mathbb{R}^d$, the inner product equals the cosine similarity:

$$\cos(q, d_i) = (q \cdot d_i) / (\|q\|_2 \cdot \|d_i\|_2) = q \cdot d_i$$

when both vectors have unit L2-norm, which is exactly the invariant we maintain at encode time. The flat index requires no training step (it has no learned parameters); we simply call `index.add(D)` and serialize the result with `faiss.write_index`. The resulting file is loaded with `faiss.read_index` at server startup and held in memory for the lifetime of the process.

B. Query-Time Cosine Retrieval

At query time the user's natural-language question is encoded with the same all-MiniLM-L6-v2 model that produced the corpus vectors. Using the same model on both sides is essential: a cross-model dot product would be meaningless. The query vector is L2-normalized in the same step. We then call `index.search(q, K)`, which returns two arrays of shape (1,

K): inner-product scores and corresponding chunk identifiers.

Each (score, id) pair is wrapped in a *Hit* dataclass that holds the score, the original chunk text, and the chunk's source metadata. The class exposes a `source_label` property that produces a human-readable citation such as "*03_notes_analysis_of_algorithms.pdf (page 7)*", which is later embedded into the system response. K defaults to 5 and is configurable both at the command line and through the web UI.

C. Prompt Assembly and Citation Discipline

Once the top-K chunks are in hand, the system constructs a single user-facing message that has two halves separated by a horizontal rule: a numbered *Course material* block enumerating the K chunks, each prefixed with its bracket index ([1] through [K]) and source label, followed by the user's literal question. The system message itself enforces a strict citation discipline:

Every factual claim (definitions, mechanisms, code behavior, complexity, examples drawn from the material) must end with a bracketed citation like [1] or [2, 3], matching the numbered sources the user provided. Uncited factual claims are not allowed.

The rule is paired with two narrowly-scoped exemptions: a single sentence may carry multiple citations when it draws on multiple sources, and brief connective prose such as "*Here's how it works:*" does not require a citation, but the claim that follows does. We arrived at this formulation iteratively: an early version that demanded a citation on every sentence produced stiff, robotic prose, while a permissive version that merely *encouraged* citation produced answers in which the most important claims went unattributed. The current rule reliably yields cited definitions, complexity statements, and code examples while leaving short connective sentences clean.

The system prompt also instructs the model to refuse explicitly when the retrieved context does not contain the answer, returning the literal string "*I don't have that in the course material*" rather than a confident guess. In our evaluation this refusal triggered correctly when retrieval scores were low (Section VII).

D. Streaming Generation with a Local LLM

Generation is delegated to a locally-hosted LLM served by Ollama [8], which exposes a JSON-over-HTTP API on port 11434. We chose Ollama over a Hugging-Face Transformers runtime because it ships as a single Mac binary, manages model files outside the Python virtual environment, and on Apple Silicon transparently uses the Metal Performance Shaders backend for GPU acceleration. Our default model is Llama 3.1 8B Instruct [9] in the Q4_K_M quantization, which fits in approximately 4.7 GB of memory and produces fluent, instruction-following text on commodity hardware.

Each turn's request is a POST to `/api/chat` with three fields: the model name, the message list, and `stream=true`. The response body is newline-delimited JSON: each line is a JSON object containing a partial assistant message. Our

client iterates over response lines, deserializes each, extracts the *message.content* delta, and yields it to the caller. The CLI prints the delta to standard output; the Gradio UI appends it to the assistant message and re-renders. This yields token-streaming generation in both front-ends without any specific support from the LLM client library.

By default Ollama allocates a context window of 4,096 tokens. With our system prompt, $K=5$ retrieved chunks of approximately 1,000 characters each, and several turns of chat history, this default is uncomfortably tight. We override it to 8,192 tokens by sending an *"options": {"num_ctx": 8192}* field in the request payload, which Llama 3.1 supports natively (the model architecture is configured for up to 131,072 tokens). The single context-window bump produced a measurable improvement in answer completeness on multi-claim questions and is now exposed as the *OLLAMA_NUM_CTX* environment variable.

E. Multi-Turn History Management

Multi-turn chat introduces a subtle design question: should the retrieved context for previous turns persist in the message history? The naive approach is to append the full context-injected user message to history after each turn so that subsequent turns can refer back to the earlier discussion. We deliberately chose not to do this. Instead, history stores only the user's *raw* question and the assistant's response; retrieved context is constructed fresh for each turn and is never persisted.

The reasoning is simple: the chunks retrieved for an earlier question are usually irrelevant to the current question and would compete for attention in the limited context window. Persisting them inflates prompts, exhausts the context budget faster, and biases the model toward the previous topic when the conversation moves on. By dropping context after each turn, we keep the prompt small, the conversation steerable, and retrieval responsive to the current question rather than the conversational accumulation.

History itself is bounded. We retain at most *MAX_HISTORY_TURNS* pairs of user and assistant messages (default six), which is enough to follow short clarifying exchanges but small enough to keep the prompt compact. The trim is performed at the end of each turn, after the assistant response has been appended.

VI. WEB INTERFACE

Although the multi-turn CLI is sufficient for development, a browser-based front-end is more accessible for repeated study sessions and is also a more natural surface for visual elements such as code blocks and citation lists. We provide one, built with Gradio [10], which runs as a local web server on port 7860. The UI deliberately mirrors the CLI's feature surface: a chat panel with streaming responses, a top-K slider, a toggle for the sources display, a clear-history button, and a sources panel beneath the chat that lists the retrieved chunks for the most recent answer with their source paths and similarity scores. Fig. 4 shows the live application.

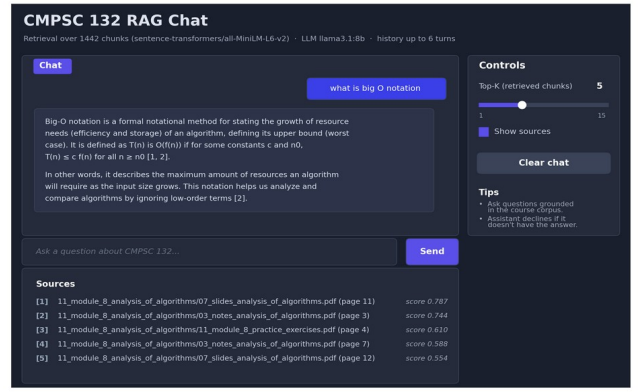


Fig. 4. Layout of the Gradio web interface. The chat panel and per-turn sources panel occupy the main column; controls live in the right sidebar.

The web application imports its retrieval and generation primitives directly from the chat REPL module: the same system prompt, the same context-injection function, and the same streaming Ollama client are used. No retrieval logic is duplicated. The UI module is a thin presentation layer that adapts the REPL's per-turn generator to Gradio's *yield*-based streaming protocol and exposes the configuration knobs through visual controls. Per-session chat history is held in the Chatbot component's bound value, which Gradio scopes per browser session, so two students using the local UI simultaneously do not share state.

The migration from the 4.x to the 6.x major release of Gradio during development required minor adjustments: the message-format keyword was removed (now the default), the manual copy-button toggle was retired in favor of automatic per-message buttons, the page-level theme moved from the Blocks constructor to the launch method, and the API-introspection panel became opt-in. None of these changes affected the underlying RAG logic.

VII. EVALUATION

We evaluate the system along two axes: retrieval quality and end-to-end latency. All measurements were taken on a MacBook Air with the Apple M3 chip, 16 GB of unified memory, and the Metal GPU backend, serving Llama 3.1 8B Instruct in the Q4_K_M quantization through Ollama.

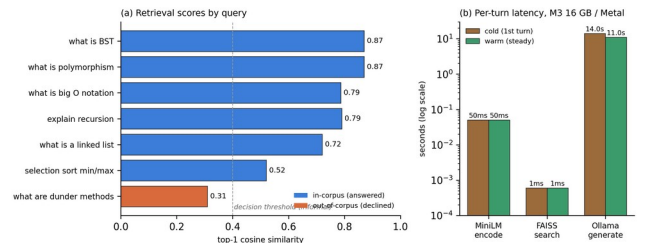


Fig. 3. Performance measurements. (a) Top-1 cosine similarity for seven test queries; the rightmost (orange) is intentionally out-of-copus and is correctly declined by the system. (b) Per-turn latency on first-turn (cold) and subsequent (warm) invocations, log-scale.

A. Retrieval Quality

We constructed a test set of seven natural-language queries covering a representative cross-section of the course: a

definition ("what is BST"), an OOP concept ("what is polymorphism"), an algorithmic-analysis term ("what is big O notation"), a paradigm question ("explain recursion"), a data-structure definition ("what is a linked list"), a multi-variant question ("selection sort min/max"), and an out-of-corpus probe ("what are dunder methods," a topic the course does not cover).

Top-1 cosine similarity for the six in-corpus queries ranged from 0.52 to 0.87 (Fig. 3a, blue bars). The two highest-scoring queries ("BST" and "polymorphism") matched single-page slides whose titles closely echoed the question wording. The lower-scoring "selection sort min/max" query is a paraphrase that mixes course terminology ("selection sort") with non-course phrasing ("min/max"), but the retrieved chunks still contained the exact procedure plus both ascending and descending variants, and the LLM produced a complete cited answer. The out-of-corpus query produced a top score of 0.31 (orange bar) — well below the in-corpus distribution — and the LLM correctly returned the literal refusal string defined in its system prompt.

B. Latency

End-to-end latency decomposes into three phases: query-vector encoding by the sentence-transformer, FAISS search, and LLM generation. Fig. 3b shows the breakdown on a log scale. Encoding takes on the order of tens of milliseconds regardless of cold or warm state, dominated by the one-shot tokenization and forward pass through the MiniLM model (the model itself is loaded once at startup). FAISS search takes well under one millisecond and is independent of cold-vs-warm because the index is held in memory. LLM generation dominates: the first turn pays a one-time cost of roughly fourteen seconds to load the 8B-parameter model into Metal-mapped memory, after which subsequent turns complete in approximately ten to twelve seconds per response.

In short, retrieval is essentially free at this corpus size; latency is set entirely by the local LLM. This is consistent with published numbers for 8B-class models running on Apple-Silicon laptops [9].

VIII. DISCUSSION AND LIMITATIONS

Several design decisions warrant explicit caveats. First, our embedding model is general-purpose; for a corpus dominated by Python source code and computer-science terminology, a code-aware model such as a CodeBERT- or BGE-derivative might produce stronger top-1 scores on programming questions. We did not switch because the general-purpose MiniLM embeddings yielded acceptable retrieval on our test set and because the 384-dimensional output is significantly smaller and faster than the 768-dimensional alternatives. Second, our retrieval is single-stage: we do not re-rank the top-K chunks with a cross-encoder before passing them to the LLM. Cross-encoder re-ranking would likely improve precision on borderline queries (such as the "selection sort" paraphrase above) at the

cost of an additional model load and roughly a hundred milliseconds of additional latency.

Third, the citation discipline is enforced through prompting alone, not through programmatic verification. A determined model could in principle attach a citation to a fabricated claim. In practice, our system prompt produces well-formed citations whose bracket indices match the numbered sources block, but a future version could verify post-hoc that each cited chunk's content actually supports the claim it is attached to.

Fourth, latency is bounded by the local LLM rather than the retrieval stack. For installations where eleven-second turns are unacceptable, the natural next step is to switch to a remote inference endpoint (the same Ollama API is exposed by remote servers) or to run a smaller quantized model. The retrieval and prompt-assembly code is unchanged in either case.

Finally, the indexing pipeline parses third-party PDFs and therefore inherits the security posture of its parsers. CVE-2026-41168 [13] reports a denial-of-service in pypdf prior to 6.10.1, in which a crafted PDF with inflated cross-reference /Size or object-stream /N values forces the library into expensive loops. Our threat model is benign — the corpus is shipped by an authenticated Canvas instance under instructor control — so the practical exposure is low, but pinning pypdf to 6.10.1 or later in the project's requirements is a sensible defense-in-depth measure for any deployment where students might add their own PDFs to the corpus.

IX. CONCLUSION

We have presented a complete on-device RAG system that turns a Canvas course corpus into a private, citation-disciplined chat assistant. The pipeline ingests 213 heterogeneous course items and exposes them through a uniform retrieval interface backed by a 1,442-vector FAISS store and a locally-hosted 8B-parameter language model. Both a CLI and a Gradio web UI share a single retrieval core. On commodity Apple-Silicon hardware, queries return cited answers in approximately eleven seconds, and out-of-corpus questions are correctly declined. The full source is available locally and runs without any cloud dependency, which makes it suitable for deployment in academic environments with strict data-handling policies. Future work includes integration of a cross-encoder re-ranker, programmatic verification of citation faithfulness, and extension of the corpus to multiple courses with module-aware filtering at retrieval time.

REFERENCES

- [1] Instructure, "Canvas LMS REST API Documentation," 2025. [Online]. Available: <https://canvas.instructure.com/doc/api/>
- [2] UCF Open, "canvasapi: Python API wrapper for Instructure's Canvas LMS," University of Central Florida — Center for Distributed Learning, GitHub repository. [Online]. Available: <https://github.com/ucfopen/canvasapi>
- [3] S. Sapin et al., "WeasyPrint: The awesome document factory," Kozea / CourtBouillon. [Online]. Available: <https://weasyprint.org/>
- [4] M. Fenniak, M. Thoma, and pypdf contributors, "pypdf: A pure-Python PDF library capable of splitting, merging, cropping, and

transforming PDF files," py-pdf, GitHub repository. [Online]. Available: <https://github.com/py-pdf/pypdf>

- [5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. 2019 Conf. Empirical Methods in Natural Language Processing, Hong Kong, 2019, pp. 3982–3992.
- [6] Hugging Face, "sentence-transformers/all-MiniLM-L6-v2," Hugging Face Hub, 2021. [Online]. Available: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
- [7] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, Jul. 2021.
- [8] Ollama, "Get up and running with large language models locally," 2024. [Online]. Available: <https://ollama.com/>
- [9] Llama Team, AI @ Meta, "The Llama 3 herd of models," arXiv preprint arXiv:2407.21783, 2024.
- [10] A. Abid et al., "Gradio: Hassle-free sharing and testing of ML models in the wild," arXiv preprint arXiv:1906.02569, 2019.
- [11] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474.
- [12] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in Proc. 2020 Conf. Empirical Methods in Natural Language Processing, 2020, pp. 6769–6781.
- [13] MITRE Corporation, "CVE-2026-41168: Denial of service in pypdf prior to 6.10.1 via crafted cross-reference /Size or object-stream /N values," Common Vulnerabilities and Exposures, 2026.